2024 IEEE Conference on Dependable, Autonomic and Secure Computing (DASC) | 979-8-3315-2272-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/DASC64200.2024.00014

Leveraging Generative AI to Enhance Automated Vulnerability Scoring

Seyedeh Leili Mirtaheri
University of Calabria
Rende, Italy
leili.mirtaheri@dimes.unical.it

Andrea Pugliese
University of Calabria
Rende, Italy
andrea.pugliese@unical.it

***Abstract*—Vulnerability assessment is an important and well-studied subject in software security. Traditional methods use expert knowledge, which is time-consuming. Considering the constantly increasing number of vulnerabilities, automated machine learning (ML)-based solutions have been proposed to assess the severity of vulnerabilities. Existing methods concentrate on predicting the Common Vulnerability Scoring System (CVSS) score or its vector metrics using available vulnerability information. The quality and diversity of the vulnerability description data can greatly affect the accuracy of these predictions. Studies report that less than 60% of such descriptions follow the formal template. On the other hand, the performance of ML-based vulnerability scoring approaches is highly dependent on the quality of the data and the model's architecture. In this paper, we aim to improve the performance of existing ML-based solutions in vulnerability assessment. We use generative artificial intelligence (AI) and feed the CVSS descriptions to a large-language model. We use GPT3.5Turbo to generate descriptions and propose a fine-tuned BERT-CNN model to predict the CVSS vector metrics. We conduct several experiments to assess the performance of the proposed method against the state-of-the-art. We use both the original dataset (6,370 descriptions) and the descriptions generated by GPT3.5Turbo. Our experiments show that our proposed architecture considerably improves accuracy.**

***Keywords-component*—Vulnerability; Severity; Prediction; Generative AI; Convolutional Neural Networks(CNN); ChatGPT; Bert; Long Short- term Memory (LSTM); TextRNN-ATT**

I. Introduction

The number of vulnerabilities has been rising consistently, increasing the need for advanced solutions. Addressing software security concerns demands prioritizing the resolution of vulnerabilities. This requires experts to conduct an advanced assessment of vulnerability severity and dealing with the most urgent ones first [1].

The national infrastructure advisory council (NIAC) has established the common vulnerability scoring system (CVSS) scoring framework for assessing the severity of vulnerabilities. Different versions of CVSS have been introduced including CVSS-v2, CVSS-v3, CVSS-V3.1, and CVSS-V4. This scoring solution enables the possibility of prioritizing the vulnerabilities [2]. CVSS basic metrics, including access vector, access complexity, authentication, confidentiality impact, integrity impact, and availability impact, are evaluated by experts for each vulnerability and finally, the severity of the vulnerability is calculated. This manual process is performed by experts in NIAC and takes about 132 days [3], [4]. As a result, manual examination of vulnerabilities is time-consuming.

Studies show that in a significant number of cases, the vulnerabilities have been exploited before receiving the severity scoring [5]. Exploiting vulnerabilities, especially critical ones, can cause irreparable damage. Therefore, utilizing automated solutions seems essential, also considering that they can provide an estimate of the severity [2].

Several studies aim to automate vulnerability assessment through ML-based methods by using the textual data available about vulnerabilities [6]. ML-based methods provide the opportunity of estimating the severity as soon as a vulnerability is published. Besides minimizing the time window for initial evaluation, ML-based methods help experts prioritize and make recommendations for their assessments.

ML-based automated approaches could generally be categorized into two groups. The first group focuses on predicting CVSS vector metrics from vulnerability descriptions. For instance, Han et al. [7] developed a system to classify vulnerabilities into different severity levels based on CVSS metrics, while Khazaei et al. [8] predicted the numerical CVSS severity score. The second group aims to predict the entire CVSS score. Elbaz et al. [1] proposed a method to automatically predict the entire CVSS vector, and Kuehn et al. [2] introduced a deep Learning (DL) system for this purpose, requiring expert-labeled data, which increases the effort for large datasets. In this research, we focus on the first category and use the description of vulnerabilities to predict the CVSS vector metrics. Since the quality of the descriptions highly affects the accuracy of the learning model, we aim to improve the quality of such descriptions.

The development of Generative AI has provided new potential in different domains. Concerning cybersecurity, Generative AI can be applied to assist researchers in developing innovative techniques to mitigate variety of threats [9]. Systems such as ChatGPT, Gemini, Claude Models, and Llama have shown tremendous potential in understanding and generating human language content [10], [11]. They offer advantages by enhancing security measures [12], [13], more particularly in this case, providing templates for vulnerability descriptions by exploring their full specifications. Generative AI such as ChatGPT has demonstrated promising results in tackling complex cybersecurity tasks [9]. Moreover, they can assist

979-8-3315-2272-8/24/$31.00 ©2024 IEEE
DOI 10.1109/DASC64200.2024.00014

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.

domain experts in their challenging task of identifying high-risk vulnerabilities and accelerate the process of evaluating the severity of these vulnerabilities, by processing descriptions and capturing the key details. Exploring Generative AI applications in cybersecurity can lay the foundations for further model utilization and transformative impacts. In addition, the traditional approaches in cybersecurity usually struggle to keep pace with evolving threats – Generative AIs is a valuable asset in this domain [9].

A. Background

CVSS v3.1 has defined six assessment metrics for determining the severity level of a vulnerability, including access vector, access complexity, authentication, confidentiality impact, integrity impact, and availability impact. These metrics are ultimately weighted and used to calculate the severity score, according to a specific formula defined by CVSS. The corresponding severity levels are established by categorizing the severity scores into various ranges.

Generally, the CVSS score considers three sets of metrics including *Base*, *Temporal*, and *Environmental* groups. The Base metrics consist of three elements, namely *exploitability*, *scope*, and *impact*. Exploitability includes attack vector, attack complexity, privileges required, and user interaction. Scope determines if the vulnerability can propagate in a system. Impact measures confidentiality, integrity, and availability.

Temporal metrics are defined to characterize vulnerabilities with a dynamic nature and include exploit code maturity, remediation level, and report confidence. Environmental metrics enable the modification and adjustment of base metrics. More information is available on the NVD website.[1]

B. Problem Statement

Reviewing the existing descriptions, we see a growing number of descriptions deviating from the template of common vulnerabilities and exposures (CVE). The number of vulnerability descriptions that adhere to templates has declined over time – 99.6% to 58.5% from 2010 to 2021 [14]. Full vulnerability descriptions contain specific elements that are critical in identifying different aspects of vulnerability in a system. However, varied descriptions without a specific template may lead to ineffective and inefficient assessments, making it difficult to acutely identify the severity of vulnerabilities. Such descriptions may vary over time, due to changes in the language and the description formats. Following an effective template that includes key vulnerability elements, while maintaining a description pattern, may be helpful in sustaining a robust approach towards accurately assessing vulnerabilities in software systems.

In this paper, we use the description of vulnerabilities to predict the CVSS vector metrics. We focus on *Base* metrics, which provide characteristics of vulnerabilities, independent of real-world exploits or mitigating controls. We aim to explore the ability of Generative AI to regenerate descriptions adhering to the standard CVE template. This is because the performance of ML-based methods for automated vulnerability scoring systems is highly dependent to the quality of data and model's architecture. We also examine the effectiveness of different ML models in characterizing vulnerability descriptions and assessing various metrics, based on descriptions either adhering to templates or the original full descriptions. We use the specified official template of CVSS to fine tune the GPT3.5Turbo model to generate the vulnerability descriptions.

[1] https://nvd.nist.gov

C. Our Contributions

Motivated by the aforementioned statements, we propose a novel approach that integrates the usage of Generative AI in generating the vulnerability description templates. We better capture the vulnerability elements in descriptions and enhance the accuracy of identifying high-risk vulnerabilities. First, we use fine-tuned GPT3.5Turbo model to extract vulnerability key elements and generate descriptions based on a provided template, and then examine the model's ability to create assessments for each element based on the provided description. We conduct several experiments to assess the performance of the model, along with the generated descriptions. We compare the performance with different state-of-the-art classifiers that are utilized to predict the assessment metrics. Comparing the accuracy metrics, we also calculate the severity score of the achieved metrics to further examine the performance of the proposed approach.

Our main contributions are summarized as follows:

- We use the potential of LLMs to improve the quality of vulnerability description data. Using the optimized prompts on ChatGPT3.5Turbo, we create a synthetic dataset, which uses the original both in template and out-of-template descriptions.
- We propose a fine-tuned BERT-based model, called FNT-BERT-CNN, for automated vulnerability assessment.
- We perform extensive experiments to evaluate the effectiveness of our augmented dataset. We use 6370 vulnerability descriptions from the original NVD dataset. We create a synthetic in template dataset for these descriptions. The FNT-BERT-CNN model demonstrates a near-perfect accuracy in categorizing diverse vulnerability aspects and outperforms the baseline methods.

The remainder of this paper is organized as follows. In Section II we discuss the most recent related work. Section III discusses datasets and the proposed model including the Generative AI for template and ML-based assessment model. In Section IV, we present the results of our experimental assessment. Finally, Section V concludes the paper, also discussing limitations and future research directions.

II. Related Work

Software vulnerabilities can be used by attackers leading to misuse of software systems, exposing critical information and data [15], [16]. Researchers have been exploring the automation of vulnerability assessment processes using natural language processing (NLP) techniques and machine learning

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.

approaches [9], [17]. These learning-based studies, such as convolutional neural network (CNN) [7], [14], and long short-term memory (LSTM) [18] aim to predict CVSS metrics and identify public exploits using learned patterns. System vulnerability descriptions have been used to examine vulnerabilities and predict CVSS assessment metrics [6], [19].

Various studies have focused on using learning-based methods such as linear support vector machine (SVM) [20], [21], Random Forest, Regression, Decision Tree, XGBoost, Light Gradient Boosting Machine, and neural networks like multi-layer perceptron (MLP) [22]–[26]. Some studies have addressed concept drift issues in CVSS metric prediction [19], using sub-word features to draw semantic meanings from existing vocabularies [27].

They also focused on predicting privilege required and user interactions vulnerability metrics using random forest [27]. Chen et al. [28] used graph convolutional network to predict CVSS metrics on X platforms. Elbaz et al. [1] used linear regression to categorize CVSS metrics, while Gawron et al. used a single model to predict different metrics [29], [30]. Kanakogi et al. [31] used a natural language processing (NLP) approach to evaluate text embedding methods, with Doc2Vec outperforming tf-idf in finding relevant attack patterns. Other studies have used logistic and linear regression methods, deep learning models, and neural networks to predict CVSS metrics.

Generative AI models have gained attention in various research areas. Various Generative AIs have been examined to be adapted to the cybersecurity domain, such as cybersecurity-enhanced models of RepairLlama [32] and Hackmentor [33], as well as GPT-based models which can be utilized in this filed through prompt engineering and guiding the general model towards it [34]. Using these models, researchers tend to adapt Generative AI through continual pre-training or supervised fine-tuning [9]. While the former aims to improve the model's understanding and application of domain knowledge, the latter focuses on enhancing task-related performance. Continual pre-training involves further training of pre-trained Generative AI using a large volume of unlabeled domain-specific data to improve their broad applicability in cybersecurity. This approach is based on the assumption that even after extensive pre-training, models can be further enhanced for specific domains or tasks. On the other hand, supervised fine-tuning uses labeled data to directly optimize the model's performance on particular cybersecurity tasks. This method is also known as instruction fine-tuning, and is particularly important for Generative AIs like ChatGPT, which are designed to follow user instructions and stay on a specific task.

Despite the advancements in machine learning and generative AI, there is still a significant gap in the literature regarding the combination of these technologies to improve the quality and consistency of vulnerability descriptions. The quality and diversity of textual data can significantly affect the accuracy of predictions.

III. Proposed Model

Figure 1 illustrates the overall architecture of the proposed model, called FNT-BERT-CNN. The model consists of four key phases, including I) the integration of Generative AI (GPT3.5Turbo) and II) regenerating the template-based vulnerability descriptions, III) utilizing the fine-tuned BERT tokenizer, and IV) evaluating the descriptions using a CNN deep learning model. We first generate a synthetic dataset using the GPT3.5Turbo model to create data according to the official CVE template. We use a BERT model, fine-tuned on the complete NVD original dataset, to each CVSS parameter individually. We utilize the results as the tokenizer, and use CNN classifier to predict the labels for each vulnerability metric. Finally, the overall CVSS score is calculated using the standard CVSS formula.

The following sections discuss each phase in more detail.

A. Original Dataset

The dataset used in this paper is from the National Vulnerability Database (NVD), specifically pertaining to software vulnerabilities. We performed a preprocessing procedure to enhance the quality and reliability of this dataset, involving meticulous data cleaning and error correction. We collected a dataset from January 2016 to February 2024 that includes more than 137,500 vulnerability descriptions, considering as metrics the attack vector (AV), attack complexity (AC), privileges required (PR), user interaction (UI), confidentiality (C), scope (D), integrity (I), and availability (A), along with their values, severity score, and CVSS score.

B. Generating Standard Dataset

To regenerate vulnerability descriptions that follow the standard CVE-specified template, a Generative AI (GPT3.5Turbo) is fine-tuned using samples extracted from the original dataset. In this respect, we have randomly sampled 120 items of CVSS vulnerability descriptions presented in the original dataset and created revised descriptions according to the official CVE template, as shown below. We have manually generated correct description samples for these fine-tuning data, ensuring that the descriptions all adhere to the defined template. Then, the GPT3.5Turbo model is fine-tuned on this data, considering 10 epochs, and the batch size of 1.

CVE Template: "PROBLEM TYPE" in "AFFECTED COMPONENTS" causes "IMPACT" when "ATTACK VECTOR"

Once the Generative AI model is fine-tuned, FNT-BERT-CNN proceeds to regenerate vulnerability descriptions adhering to the official template. In order to direct the model toward the desired template, we incorporated the following system message and prompt. The model's temperature is set to 1 and the max token is specified as 500 for all the application programming interface (API) calls.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.

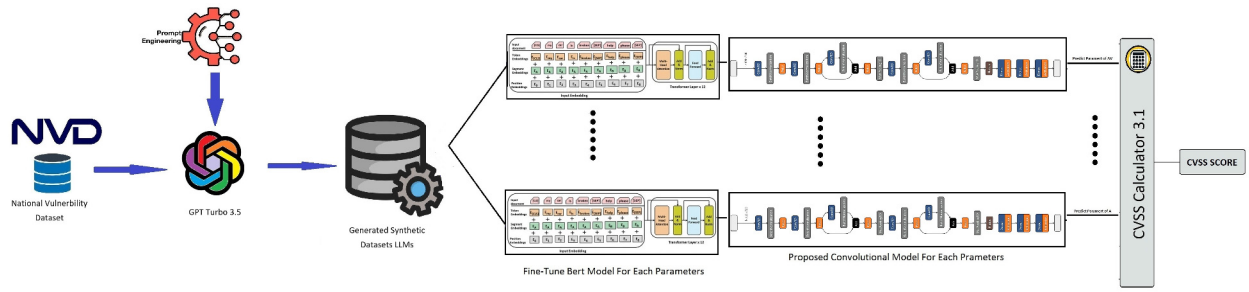


Fig. 1: An overview of the proposed FNT-BERT-CNN method's architecture.

System Message: "You are given the CVE description of a software vulnerability. You should rephrase the Descriptions following this template:
"PROBLEM TYPE" in "AFFECTED COMPONENTS" causes "IMPACT" when "ATTACK VECTOR"
Please generate a comprehensive and formatted description, containing the attack type, impact of the vulnerability, the software component affected, and any attack vectors for the given CVE based on the mentioned template and description."

Prompt: "The CVE description for "{cve_id}" is as follows: "{description}"
Please generate a comprehensive and reformatted description for this CVE based on the following template. The [DESCRIPTION] field should include an explanation of the attack type, the impact of the vulnerability, the software components affected, and any attack vectors. Do NOT explain and just stick to the given template.
Descriptions often follow this template:
"PROBLEM TYPE" in "AFFECTED COMPONENTS" causes "IMPACT" when "ATTACK VECTOR" "

C. Vulnerability Assessment Model

Once the template-based descriptions are generated, the model proceeds with the vulnerability assessment process. The following procedure entails applying the BERT tokenizer to the generated synthetic data. For this purpose, we have employed the pretrained *BERT-base-uncased* BERT as our primary tokenizer model. Subsequently, we fine-tuned the BERT model based on the original NVD dataset, while excluding the sampled data encompassing a vulnerability distribution of 6,370 instances. Once the BERT tokenizer has been fine-tuned using the vulnerability descriptions, the fine-tuned BERT model is utilized as the tokenizer within our model.

The BERT tokenizer is also fine-tuned on the all collected dataset corresponding to each metric separately, aiming to effectively encapsulate the nuances in the descriptions linked to that specific metric. The rationale behind this approach lies in the fact that various metrics offer insights into different facets of a system, such as problem types or the impact on the affected system [14]. We augmented the pre-trained BERT tokenizer model and comprehensively captured the intricacies present in the descriptions associated with a particular metric. This, in turn, leads to more accurate predictions from the final model.

We have incorporated the TextCNN model within our architecture. This model is applied to the tokenized instances of the dataset, respecting to each metric individually. Consequently, each vulnerability description undergoes examination based on a specific metric, and the model predicts its corresponding value according to valid metric values. Subsequently, we employ the CVSS severity score evaluator to further analyze the vector of diverse metrics for a given vulnerability, ultimately calculating its severity score based on the individual metric values.

IV. Experimental Results

This section discusses the analysis carried out to assess the impact of utilizing template-based Generative AI-generated vulnerability descriptions, and evaluates the performance of different state-of-the-art baseline models, in comparison to FNT-BERT-CNN.

A. Data

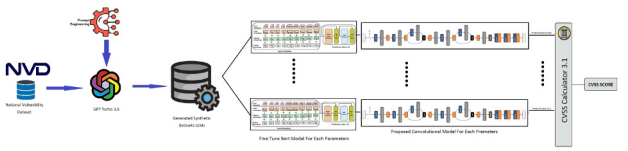
The experiments involved analyzing a dataset of 6,370 vulnerability descriptions, preserving their probability distribution and ratio as observed in the original dataset. This ensures that network training and evaluation are executed on a clean and well-balanced dataset, accurately representing the original data's fundamental characteristics and ensuring accurate label distribution.

B. Baseline Methods

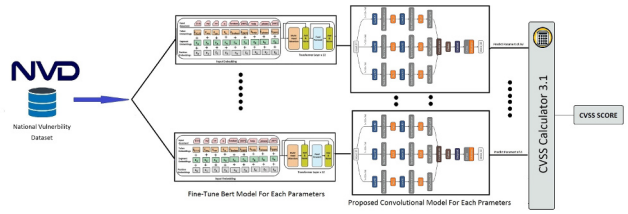
To evaluate the performance of the proposed model, we have compared FNT-BERT-CNN to BERT-LSTM, BERT-TextRNN-ATT, and BERT-Text-CNN [14]. Figure 2 illustrates the architectural overview of these approaches.

The BERT-LSTM and BERT-TextRNN-ATT models are similar in architecture, using pre-trained BERT tokenizers and

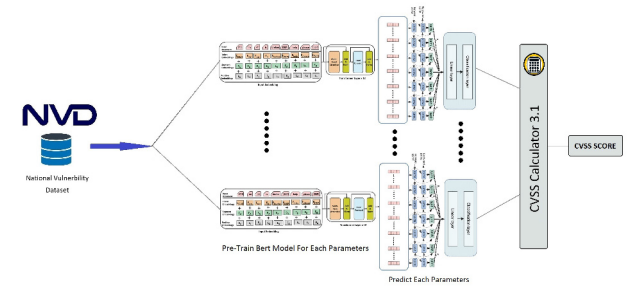
Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.



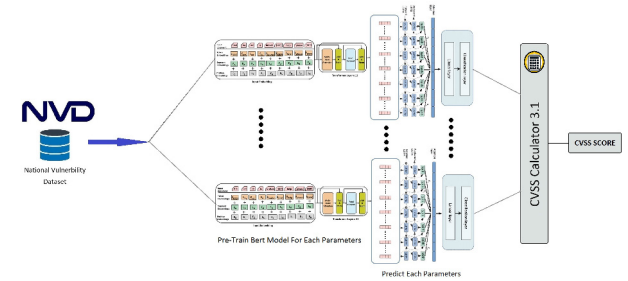
(a) FNT-BERT-CNN



(b) BERT-Text-CNN



(c) BERT-LSTM



(d) BERT-LSTM-ATT

Fig. 2: Architectural overview of the baseline and proposed models.

LSTM layers, just there is an extra attention layer in BERT-TextRNN-ATT. The BERT-Text-CNN model uses 1D and 2D CNN architecture, pooling, batch normalization layers, and a Generative AI component for template-refined vulnerability descriptions. FNT-BERT-CNN uses a Generative AI component to capture key elements of a description without relying on peripheral details. The model uses a BERT tokenizer fine-tuned on the lexical content of the dataset, excluding the data used for evaluation. The input goes through two layers of Relu (Batch Normalization – Conv1D) with input and output channels of (768, 32) and (32, 64), respectively, with a kernel size of 3 and padding. The data undergoes a Max pooling layer with a kernel size of 2, a Relu (Batch Normalization – Conv1D) layer with 64 channels, a Max pooling layer with a kernel size of 2, and a Max pooling layer. The output is flattened, goes through two linear layers with Relu activation functions, and a single output layer for classification.

Likewise, the BERT-TextRNN-ATT model (Figure 2d) consists of a similar architecture, with a single attention layer following the LSTM processing layer and before the dropout. The attention layer is initialized using the Xavier uniform initialization and is constructed using the same dimension of the hidden layer.

BERT-LSTM and BERT-TextRNN-ATT models were trained using the *cross entropy* loss function, *AdamW* optimizer, with a learning rate of 10^{-4}, a weight_decay of 10^{-5}, and an Exponential LR with gamma set to 0.99. On the other hand, FNT-BERT-CNN and BERT-Text-CNN are trained using a modified cross-entropy loss function, named *Focal Loss* with gamma set to 2, and similar optimizer settings. All the models were trained for 20 epochs, with 80% of data used for training purposes, and 20% for evaluation.

C. Evaluation Metrics

The study evaluates classification models using metrics like Macro F1-score, Macro Precision, Macro Recall, Accuracy, and Cohen Kappa Score. Accuracy measures the proportion of accurate predictions, F1-score combines precision and recall, Macro Recall evaluates recall for each class, Macro Precision determines precision, and Cohen Kappa score evaluates agreement between observed and expected accuracy.

D. Impact of Generated Descriptions

This section explores the impact of generated descriptions using the Generative AI (fine-tuned GPT3.5Turbo), in comparison to the models' performances on the original data. Utilizing the fine-tuned GPT3.5Turbo model, we have generated vulnerability descriptions based on the defined template, capturing the key element in the descriptions. Table I illustrates the values obtained for each vulnerability parameter, using each of the experimented models, on two datasets of original data, and template-based regenerated descriptions.

As Table I shows, in general, the GPT3.5Turbo generated descriptions outperformed the original dataset, leading to more accurately predicted parameters. For instance, looking at the BERT-LSTM model, the GPT3.5Turbo generated descriptions enhanced the prediction accuracy, recall, and F1-score for the six parameters of *PR, UI, S, C, I, A*, resulting in the highest accuracy metrics. According to the results, the generated descriptions led to 0.78, 0.87, 0.95, 0.80, 0.84, and 0.86 accuracy for the mentioned parameters, respectively, whereas the values of this metric for the original data were 0.76, 0.85, 0.93, 0.77, 0.78, 0.84. This shows an improvement of over 2% in all the cases and a maximum improvement of 6%. On the other hand, the original data resulted in merely 2 parameters, *av* and *ac,* with the highest values, in terms of the mentioned metrics. Notably, the generated refined descriptions led to the highest precision values in all the parameters, showing the accuracy

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.

in predicting the correct class labels for the parameters, when the descriptions follow a specific template.

TABLE I: Impact of generating template-based vulnerability descriptions on the performance of the experimented models; A: Accuracy, P: Precision, R: Recall, F1: F1-score, C: Cohen Kappa.

Original Data										
	BERT-LSTM					**BERT-Text-CNN**				
	A	R	P	F1	C	A	R	P	F1	C
AV	**0.851**	**0.431**	0.531	**0.686**	**0.622**	**0.895**	0.548	**0.847**	**0.618**	**0.719**
AC	**0.924**	**0.617**	0.843	**0.663**	**0.337**	0.942	0.716	0.898	0.774	0.552
PR	0.761	0.548	0.623	0.572	0.452	0.838	0.637	**0.801**	0.683	0.629
UI	0.858	0.837	0.848	0.842	0.684	**0.918**	**0.892**	**0.931**	**0.907**	**0.814**
S	0.935	0.839	0.916	0.871	0.743	**0.968**	**0.908**	**0.977**	**0.938**	**0.876**
C	0.779	0.694	0.785	0.727	0.581	**0.845**	**0.795**	0.842	**0.815**	**0.717**
I	0.786	0.747	0.799	0.768	0.637	0.854	0.825	0.870	0.844	0.754
A	0.842	0.572	0.889	0.564	0.677	0.875	0.607	0.746	0.608	0.745
	BERT-LSTM-ATT					**FNT-BERT-CNN**				
	A	R	P	F1	C	A	R	P	F1	C
AV	0.861	**0.471**	**0.810**	**0.535**	0.608	0.993	**0.995**	0.971	**0.983**	**0.983**
AC	**0.942**	**0.728**	**0.883**	**0.781**	**0.565**	**0.995**	0.982	0.990	0.986	0.971
PR	**0.812**	0.649	0.690	0.666	**0.592**	**0.986**	0.968	**0.980**	0.974	**0.970**
UI	**0.900**	**0.874**	**0.904**	**0.886**	**0.773**	0.990	0.987	0.991	0.989	0.978
S	**0.961**	**0.900**	0.956	**0.925**	**0.850**	0.992	0.984	0.988	0.986	0.972
C	0.797	0.739	**0.778**	0.756	0.632	0.984	0.980	0.984	0.982	0.972
I	0.819	0.814	0.806	0.809	0.701	**0.983**	0.979	0.983	0.981	0.972
A	0.859	**0.597**	**0.650**	0.608	0.707	0.985	0.968	0.979	0.974	0.970
GPT3.5Turbo										
	BERT-LSTM					**BERT-Text-CNN**				
	A	R	P	F1	C	A	R	P	F1	C
AV	0.827	0.405	**0.765**	0.413	0.549	0.888	**0.548**	0.829	0.610	0.707
AC	0.918	0.567	**0.841**	0.595	0.212	**0.951**	**0.742**	**0.912**	**0.800**	**0.603**
PR	**0.782**	**0.563**	**0.654**	**0.589**	**0.503**	**0.839**	**0.685**	0.790	**0.724**	**0.633**
UI	**0.870**	**0.847**	**0.865**	**0.855**	**0.710**	0.911	0.887	0.916	0.899	0.799
S	**0.954**	**0.871**	**0.960**	**0.908**	**0.817**	0.965	0.905	0.968	0.933	0.866
C	**0.801**	**0.722**	**0.807**	**0.754**	**0.627**	0.838	0.760	**0.864**	0.799	0.689
I	**0.840**	**0.812**	**0.853**	**0.830**	**0.730**	**0.879**	**0.851**	**0.897**	**0.870**	**0.795**
A	**0.863**	**0.583**	**0.906**	**0.578**	**0.716**	**0.878**	**0.607**	**0.749**	**0.609**	**0.750**
	BERT-LSTM-ATT					**FNT-BERT-CNN**				
	A	R	P	F1	C	A	R	P	F1	C
AV	**0.862**	0.453	0.748	0.486	**0.624**	0.991	0.993	**0.971**	0.982	0.979
AC	0.940	0.719	0.880	0.773	0.549	**0.998**	**0.987**	**0.999**	**0.993**	**0.986**
PR	0.803	**0.659**	**0.709**	**0.680**	0.565	0.983	**0.974**	0.973	**0.974**	0.964
UI	0.891	0.868	0.891	0.877	0.755	**0.992**	**0.990**	**0.992**	**0.991**	**0.983**
S	0.958	0.881	**0.965**	0.916	0.833	**0.997**	**0.994**	**0.994**	**0.994**	**0.989**
C	**0.819**	**0.828**	0.744	**0.776**	**0.660**	**0.988**	**0.987**	**0.990**	**0.988**	**0.982**
I	**0.847**	**0.825**	**0.853**	**0.837**	**0.745**	0.991	**0.990**	**0.991**	**0.990**	**0.985**
A	**0.872**	0.587	0.582	**0.917**	**0.734**	**0.987**	**0.970**	**0.991**	**0.980**	**0.973**

The BERT-Text-CNN model shows superior performance using the generated template-based descriptions on the parameters *AC, PR, I*, and *A*, leading to 0.95, 0.84, 0.88, and 0.88 accuracy, respectively, improving this metric over 2% in *I*. Higher Cohen Kappa values shows more accurate and precise classification of the mentioned metrics. However, according to Table I the model demonstrated superior performance when analyzing the original data in contrast to the GPT3.5Turbo generated descriptions, resulting in higher Recall, Precision, and F1 scores. This implies that the original descriptions offered more comprehensive information to the BERT-Text-CNN model for the *AV, UI, C, S* parameters, enabling precise vulnerability classification. Overall, it is demonstrated that both the experimented data types resulted in the highest accuracy for 4 out of 8 parameters, which shows similar performance on the BERT-Text-CNN performance, in terms of the accuracy metric.

By contrast, the BERT-LSTM-Attention model exhibited a decline in accuracy when classifying vulnerability parameters of *AC, PR, UI,* and *S* using GPT3.5Turbo generated data compared to original data, while showing superior performance on the *AV, C, I,* and *A* parameters. The model also witnessed higher values in all metrics of Precision, Recall, F1-score, and Cohen Kappa, regarding the *I* parameter, and higher Recall and F1-score in *C*. However, the model achieved marginally lower Recall and Precision values on 5 out of 8 parameters, when GPT3.5Turbo generated descriptions were used compared to original vulnerability descriptions. Similarly, Cohen's Kappa values for different parameters on GPT3.5Turbo generated descriptions also are outperformed by the original data, on 4 out of 8 parameters. Lower Cohen's Kappa scores for GPT3.5Turbo generated data indicate less agreement between the model's predictions on the same data points compared to using original data. Overall, these findings highlight that the BERT-LSTM-ATT model was unable to accurately classify vulnerability parameters, using the GPT3.5Turbo generated descriptions, and showed higher performance on the original data.

Finally, the FNT-BERT-CNN achieved significantly higher accuracy with the GPT3.5Turbo generated descriptions compared to the original vulnerability descriptions for vulnerability parameter classification. It is evident across all reported metrics, including Accuracy, Recall, Precision, F1-score, and Cohen Kappa score, and for the majority of the vulnerability parameters, including *AC, UI, S, C,* and *A*. The model achieved a substantial improvement in Recall and F1-score metrics on the GPT3.5Turbo generated descriptions, for all the vulnerability parameters except *AV*. This suggests that template-based descriptions can provide crucial information for the model to accurately recognize vulnerabilities. Similarly, Precision scores were also consistently higher for the generated data across most vulnerability classes.

While the model performed well on both data types for the majority of the vulnerability parameters, generated data yielded noticeable improvements on all classification metrics, suggesting that imposing a template on the vulnerability descriptions can lead to a positive impact on the classification process. Notably, the model architecture plays a significant role in the general performance of classifying various vulnerability metrics. As it can be seen the template-based generated data enhanced the overall performance of FNT-BERT-CNN, along with the BERT-LSTM and BERT-LSTM-ATT models. The BERT-Text-CNN model was unable to yield accurate predictions.

E. Evaluating the Proposed Model

In this section, we provide a comprehensive comparison of FNT-BERT-CNN with the models under experimentation. Tables II to V present information regarding the accuracy and precision metrics, comparing the performance of FNT-BERT-CNN and the baselines models on both original and template-based generated vulnerability descriptions via GPT3.5Turbo.

Based on Tables II and III, FNT-BERT-CNN significantly outperforms all the baseline models in terms of accuracy for all vulnerability parameters (*AV, AC, PR, UI, S, C, I*, and *A*) on the original descriptions. This model yielded an accuracy of over 98% for all the parameters, and 99% for *AC, UI*, and *S*. Moreover, FNT-BERT-CNN showed consistent results

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.

on all the parameters, whereas, the baselines demonstrated inconsistent accuracy values, ranging from 70% to 94%. In contrast to FNT-BERT-CNN model, the baseline BERT-CNN could accurately classify merely around 80% of the vulnerability parameters. Furthermore, the baseline models exhibited lower overall performance, with trade-offs between different parameters. For instance, BERT-CNN achieved good accuracy (0.875) on *A* parameter, while having the lowest precision on this parameter. BERT-LSTM also suffered from lower precision (0.531) on *AV*, while its accuracy was about 0.851. These findings suggest that FNT-BERT-CNN is a more robust and accurate solution for classifying vulnerability parameters, yielding the highest accuracy, as well as precision metrics for all the parameters.

TABLE II: Comparison of different models' performance on the original descriptions, in terms of accuracy.

	AV	AC	PR	UI	S	C	I	a
BERT-CNN	0.895	0.942	0.838	0.918	0.968	0.845	0.854	0.875
BERT-LSTM	0.851	0.924	0.761	0.858	0.935	0.779	0.786	0.842
BERT-LSTM-ATT	0.861	0.942	0.812	0.900	0.961	0.797	0.819	0.859
FNT-BERT-CNN	**0.993**	**0.995**	**0.986**	**0.990**	**0.992**	**0.984**	**0.983**	**0.985**

TABLE III: Comparison of different models' performance on the original descriptions, in terms of precision.

	AV	AC	PR	UI	S	C	I	a
BERT-CNN	0.847	0.898	0.801	0.931	0.977	0.842	0.870	0.746
BERT-LSTM	0.531	0.843	0.623	0.848	0.916	0.785	0.799	0.889
BERT-LSTM-ATT	0.810	0.883	0.690	0.904	0.956	0.778	0.806	0.650
FNT-BERT-CNN	**0.971**	**0.990**	**0.980**	**0.991**	**0.988**	**0.984**	**0.983**	**0.979**

In reference to the performance of the models concerning the template-based GPT3.5Turbo generated descriptions, Tables IV and V present the accuracy and precision outcomes of the proposed as well as baseline models, respectively. The evaluation of different classifiers on the GPT3.5Turbo generated descriptions reveals a clear performance advantage for FNT-BERT-CNN. While all models achieved good accuracy on some metrics (over 80%), FNT-BERT-CNN exhibited superior performance across all eight vulnerability parameters, in terms of Accuracy (0.991) and Macro Precision (0.983), suggesting that the proposed model generalizes well to unseen data and offers a significant improvement over baseline models. Moreover, it can also be seen that this model indicated a consistent performance of over 97% across different vulnerability parameters, whereas the baseline architectures showed extensive fluctuations, in particular to the precision metric which the resulting values range from 60% to 90%.

TABLE IV: Comparison of different models' performance on GPT3.5Turbo generated descriptions, in terms of accuracy.

Classifier	AV	AC	PR	UI	S	C	I	A
BERT-CNN	0.888	0.951	0.839	0.911	0.965	0.838	0.879	0.878
BERT-LSTM	0.827	0.918	0.782	0.870	0.954	0.801	0.840	0.863
BERT-LSTM-ATT	0.862	0.940	0.803	0.891	0.958	0.819	0.847	0.872
FNT-BERT-CNN	**0.991**	**0.998**	**0.983**	**0.992**	**0.997**	**0.988**	**0.991**	**0.987**

V. Conclusions

In this work, we explored the effectiveness of template-based GPT3.5Turbo descriptions for vulnerability parameter classification. Our analysis of evolving vulnerability descriptions informed the development of a novel approach that leverages a defined template for vulnerability descriptions instead of original descriptions, which may extensively vary among different reported cases, for severity assessment. Moreover, we have presented a novel Generative AI-based classification model, which leverages the power of fine-tuned Generative AI, as well as a fine-tuned BERT tokenizer. Experiments demonstrated that this approach led to significantly higher accuracy (99%) compared to previous BERT-based deep learning architectures. This suggests that our proposed FNT-BERT-CNN model, fine-tuned on original data and evaluated on both the original and template-adhering generated vulnerability descriptions, effectively captures the key characteristics for vulnerability classification and results in noticeably higher accuracy and precision values.

TABLE V: Comparison of different models' performance on GPT3.5Turbo generated descriptions, in terms of precision.

Classifier	AV	AC	PR	UI	S	C	I	A
BERT-CNN	0.829	0.912	0.790	0.916	0.968	0.864	0.897	0.749
BERT-LSTM	0.765	0.841	0.654	0.865	0.960	0.807	0.853	0.906
BERT-LSTM-ATT	0.748	0.880	0.709	0.891	0.965	0.744	0.853	0.582
FNT-BERT-CNN	**0.971**	**0.999**	**0.973**	**0.992**	**0.994**	**0.990**	**0.991**	**0.991**

Our analysis of the original and template-based regenerated vulnerability also illustrates the effectiveness of utilizing a certain template, which provides efficient information regarding the key elements of a description, unifying varied reported data. In this respect, utilizing a more specific and complex model, fine-tuned on the particular task at hand, can play a significant role in enhancing the accuracy and the performance of the vulnerability classification models.

A. Limitations

The proposed method of using Generative AI to standardize vulnerability descriptions and fine-tune a BERT-CNN model for vulnerability assessment shows significant improvements in accuracy and precision. However, it has limitations such as reliance on quality and representative data, substantial computational resources, reliance on a predefined template, focus on textual data, and the need for regular updates to adapt to evolving cybersecurity landscapes and emerging threats. In future works, we aim to address these limitations.

B. Future Directions

Future research aims to improve vulnerability evaluation by exploring template structures and developing automatic vulnerability assessment methods. This will provide evaluators with a baseline of vulnerability severity and aid in handling issues promptly. Future research should integrate more vulnerability parameters and correlations, generating single models that can classify different parameters, enhancing accuracy and automating the process.

Acknowledgement

This work was partially supported by project SERICS (PE00000014) under the MUR National Recovery and Re-

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.

silience Plan funded by the European Union – NextGenerationEU.

REFERENCES

[1] Clément Elbaz, Louis Rilling, and Christine Morin. Fighting n-day vulnerabilities with automated cvss vector prediction at disclosure. In *Proceedings of the 15th International Conference on Availability, Reliability and Security*, pages 1–10, 2020.

[2] Philipp Kühn, David N Relke, and Christian Reuter. Common vulnerability scoring system prediction based on open source intelligence information sources. *Computers & Security*, 131:103286, 2023.

[3] Francesco Lomio, Emanuele Iannone, Andrea De Lucia, Fabio Palomba, and Valentina Lenarduzzi. Just-in-time software vulnerability detection: Are we there yet? *Journal of Systems and Software*, 188:111283, 2022.

[4] Sofonias Yitagesu, Xiaowang Zhang, Zhiyong Feng, Xiaohong Li, and Zhenchang Xing. Automatic part-of-speech tagging for security vulnerability descriptions. In *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, pages 29–40. IEEE, 2021.

[5] Kevin Townsend. Vulnerabilities being exploited faster than ever: Analysis, February 2023.

[6] Triet HM Le, Huaming Chen, and M Ali Babar. A survey on data-driven software vulnerability assessment and prioritization. *ACM Computing Surveys*, 55(5):1–39, 2022.

[7] Zhuobing Han, Xiaohong Li, Zhenchang Xing, Hongtao Liu, and Zhiyong Feng. Learning to predict severity of software vulnerability using only vulnerability description. In *2017 IEEE International conference on software maintenance and evolution (ICSME)*, pages 125–136. IEEE, 2017.

[8] Atefeh Khazaei, Mohammad Ghasemzadeh, and Vali Derhami. An automatic method for cvss score prediction using vulnerabilities description. *Journal of Intelligent & Fuzzy Systems*, 30(1):89–96, 2016.

[9] Jie Zhang, Haoyu Bu, Hui Wen, Yu Chen, Lun Li, and Hongsong Zhu. When llms meet cybersecurity: A systematic literature review. *arXiv preprint arXiv:2405.03644*, 2024.

[10] Barret Zoph, Colin Raffel, Dale Schuurmans, Dani Yogatama, Denny Zhou, Don Metzler, Ed H Chi, Jason Wei, Jeff Dean, Liam B Fedus, et al. Emergent abilities of large language models. *TMLR*, 2022.

[11] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey. *arXiv preprint arXiv:2402.06196*, 2024.

[12] Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, page 100211, 2024.

[13] Yagmur Yigit, William J Buchanan, Madjid G Tehrani, and Leandros Maglaras. Review of generative ai methods in cybersecurity. *arXiv preprint arXiv:2403.08701*, 2024.

[14] Xiaobing Sun, Zhenlei Ye, Lili Bo, Xiaoxue Wu, Ying Wei, Tao Zhang, and Bin Li. Automatic software vulnerability assessment by extracting vulnerability elements. *Journal of Systems and Software*, 204:111790, 2023.

[15] Andy Zou, Zifan Wang, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

[16] Faheem Ullah, Matthew Edwards, Rajiv Ramdhany, Ruzanna Chitchyan, M Ali Babar, and Awais Rashid. Data exfiltration: A review of external attack vectors and countermeasures. *Journal of Network and Computer Applications*, 101:18–54, 2018.

[17] Seyedeh Leili Mirtaheri and Reza Shahbazian. *Machine learning: theory to applications*. CRC Press, 2022.

[18] Sefa Eren Sahin and Ayse Tosun. A conceptual replication on predicting the severity of software vulnerabilities. In *Proceedings of the 23rd International Conference on Evaluation and Assessment in Software Engineering*, pages 244–250, 2019.

[19] Triet Huynh Minh Le, Bushra Sabir, and Muhammad Ali Babar. Automated software vulnerability assessment with concept drift. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, pages 371–382. IEEE, 2019.

[20] Mehran Bozorgi, Lawrence K Saul, Stefan Savage, and Geoffrey M Voelker. Beyond heuristics: learning to classify vulnerabilities and predict exploits. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 105–114, 2010.

[21] Carl Sabottke, Octavian Suciu, and Tudor Dumitraș. Vulnerability disclosure in the age of social media: Exploiting twitter for predicting {Real-World} exploits. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 1041–1056, 2015.

[22] Mohammed Almukaynizi, Eric Nunes, Krishna Dharaiya, Manoj Senguttuvan, Jana Shakarian, and Paulo Shakarian. Proactive identification of exploits in the wild through vulnerability mentions online. In *2017 International Conference on Cyber Conflict (CyCon US)*, pages 82–88. IEEE, 2017.

[23] Mohammed Almukaynizi, Eric Nunes, Krishna Dharaiya, Manoj Senguttuvan, Jana Shakarian, and Paulo Shakarian. Patch before exploited: An approach to identify targeted software vulnerabilities. *AI in Cybersecurity*, pages 81–113, 2019.

[24] Daniel Alves de Sousa, Elaine Ribeiro de Faria, and Rodrigo Sanches Miani. Evaluating the performance of twitter-based exploit detectors. *arXiv preprint arXiv:2011.03113*, 2020.

[25] Jay Jacobs, Sasha Romanosky, Idris Adjerid, and Wade Baker. Improving vulnerability remediation through better exploit prediction. *Journal of Cybersecurity*, 6(1):tyaa015, 2020.

[26] Octavian Suciu, Connor Nelson, Zhuoer Lyu, Tiffany Bao, and Tudor Dumitraș. Expected exploitability: Predicting the development of functional vulnerability exploits. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 377–394, 2022.

[27] Saahil Ognawala, Ricardo Nales Amato, Alexander Pretschner, and Pooja Kulkarni. Automatically assessing vulnerabilities discovered by compositional analysis. In *Proceedings of the 1st International Workshop on Machine Learning and Software Engineering in Symbiosis*, pages 16–25, 2018.

[28] Haipeng Chen, Jing Liu, Rui Liu, Noseong Park, and VS Subrahmanian. Vest: A system for vulnerability exploit scoring & timing. In *IJCAI*, pages 6503–6505, 2019.

[29] Marian Gawron, Feng Cheng, and Christoph Meinel. Automatic vulnerability classification using machine learning. In *Risks and Security of Internet and Systems: 12th International Conference, CRiSIS 2017, Dinard, France, September 19-21, 2017, Revised Selected Papers 12*, pages 3–17. Springer, 2018.

[30] Xi Gong, Zhenchang Xing, Xiaohong Li, Zhiyong Feng, and Zhuobing Han. Joint prediction of multiple vulnerability characteristics through multi-task learning. In *2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS)*, pages 31–40. IEEE, 2019.

[31] Kenta Kanakogi, Hironori Washizaki, Yoshiaki Fukazawa, Shinpei Ogata, Takao Okubo, Takehisa Kato, Hideyuki Kanuka, Atsuo Hazeyama, and Nobukazu Yoshioka. Tracing CAPEC attack patterns from CVE vulnerability information using natural language processing technique. In *54th Hawaii International Conference on System Sciences, HICSS 2021, Kauai, Hawaii, USA, January 5, 2021*, 2021.

[32] André Silva, Sen Fang, and Martin Monperrus. Repairllama: Efficient representations and fine-tuned adapters for program repair. *arXiv preprint arXiv:2312.15698*, 2023.

[33] Jie Zhang, Hui Wen, Liting Deng, Mingfeng Xin, Zhi Li, Lun Li, Hongsong Zhu, and Limin Sun. Hackmentor: Fine-tuning large language models for cybersecurity. In *2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 452–461. IEEE, 2023.

[34] Shafi Parvez Mohammed and Gahangir Hossain. Chatgpt in education, healthcare, and cybersecurity: Opportunities and challenges. In *2024 IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0316–0321. IEEE, 2024.

Authorized licensed use limited to: b-on: Universidade de Lisboa Reitoria. Downloaded on September 27,2025 at 13:26:38 UTC from IEEE Xplore. Restrictions apply.